

**Agents Write 7x More Code.
Releases Grew 20%.**

The AI Productivity Paradox Is a QA Problem

7x code vs +20% releases

The Feeling of Speed Is Deceiving

82% of engineers spend **less time writing code** (Annie Vella study, 28 countries)

84% report higher productivity — yet experience got worse: **14% -> 27%**

GitLab: 78% code faster, **79% say delivery hasn't sped up**

METR: felt **+20%** faster, measured **19% slower**

+20% felt vs **-19% measured**

"We do more and enjoy it less."

Where the Time Went: the Middle Loop



Inner: write · build · run · fix (IDE, TDD)

Middle: Direct · Evaluate · Correct

Outer: commit · review · deploy · monitor (DevOps)

Supervising an agent is the new QA territory.

What Breaks First? The Review Pipeline

BEFORE

90% of engineers used AI → CEO saw 0 value. Org stuck at Stage 1-2.

AFTER

AI-code +69%, PRs **21x** → human review pipeline **broke**.

21x automated PRs

The fix was a QA design decision: one agent finds, another repairs.

Where QA Earns Its Keep Now

1. **AI reviewer + auto-fix loop** — find, repair, then human
2. **Regression advice on every PR** — checklists from git diff
3. **Mutation testing as anti-overfit guardrail** — 34/34 caught
4. **Comprehension debt control** — specs are the new code

Verification scales with PR volume. Trust is measured, not assumed.

Writing Got Cheap. Verifying Did Not.

Understanding, verifying, shipping, maintaining — all more expensive

AI code is dangerous because it is **plausible**: compiles, runs, logic error three commits deep

Without a verification layer, headcount cuts become **hidden tech debt**

Teams that invest in the verification layer ship.

Writing became almost free. Everything after it became more expensive.

Which Stage Is Your Team on the 0-5 Scale?

What breaks first when you hit 4?

Full analysis in the article — [link in comments](#)